Quick Notes Dashboard

BOM & DOM Lab Activity — Step-by-Step Guide

30 steps · Build from scratch · Pure JavaScript

Learning Goals

- Select DOM elements with getElementById and querySelector
- Create elements using createElement, appendChild, textContent
- Remove elements with remove() or by clearing a container
- Use BOM features: alert(), confirm(), prompt()
- Read window.innerWidth / innerHeight and the resize event
- Read navigator.userAgent
- Store data in a plain JavaScript array (no persistence)

Lab Constraint — DOM Practice Rules

Do NOT use innerHTML or insertAdjacentHTML to build note items.
Use ONLY: createElement · textContent · classList · appendChild · remove()
This forces you to practice the fundamental DOM manipulation API.

Part 1 – HTML Shell & CSS Setup

1 Create the HTML file

Create a new file called index.html. Add the standard HTML5 boilerplate below.

The <meta viewport> tag is required for mobile responsiveness.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Quick Notes Dashboard</title>
</head>
<body>

  <script>
    // JS goes here later
  </script>

</body>
</html>
```

Tip

Open index.html in your browser. You should see a blank white page — that's correct!

2 Add global CSS reset & body style

Inside <head>, add a <style> block. Paste this reset and body style first.

box-sizing: border-box makes padding predictable for every element.

```
<style>
  * { margin: 0; padding: 0; box-sizing: border-box; }
  body {
    font-family: system-ui, "Segoe UI", sans-serif;
    background: #eef2ff;
    padding: 2rem 1.5rem;
  }
</style>
```

Tip

Reload the page — the background should now be a soft indigo/lavender colour.

3 Add the .container wrapper

Add a inside <body>. Then add its CSS.

max-width + margin auto centres the layout on wide screens.

```
/* CSS */
.container { max-width: 1000px; margin: 0 auto; }

<!-- HTML inside <body> -->
<div class="container">
  <!-- all content goes here -->
</div>
```

4 Build the header card

Inside .container, add the <header> element and its styles.

The gradient trick on h1 uses background-clip: text to colour only the text.

```
<!-- HTML inside .container -->
<header>
  <h1>📝 Quick Notes Dashboard</h1>
  <p>Practice <strong>DOM manipulation</strong> + <strong>BOM</strong>.</p>
  <div class="badge">🎯 Lab: BOM + DOM | pure JS</div>
</header>

/* CSS */
```

```
header {
  background: white; border-radius: 1.5rem;
  padding: 1.5rem 2rem; margin-bottom: 2rem;
  box-shadow: 0 4px 12px rgba(0,0,0,0.05);
  border: 1px solid #cbd5e1;
}
header h1 {
  font-size: 1.8rem;
  background: linear-gradient(135deg, #3b82f6, #8b5cf6);
  -webkit-background-clip: text; background-clip: text;
  color: transparent;
}
.badge {
  background: #e0e7ff; color: #1e40af;
  padding: 0.2rem 0.8rem; border-radius: 20px;
  font-size: 0.75rem; font-weight: 600;
  display: inline-block; margin-top: 0.5rem;
}
```

Tip

Reload — you should see the gradient "Quick Notes Dashboard" heading and the blue badge.

5 Create the two-column grid

After </header>, add a .grid div. Its two children will be .card divs.

The @media rule collapses the grid to a single column on small screens.

```
/* CSS */
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1.8rem;
}
@media (max-width: 700px) {
  .grid { grid-template-columns: 1fr; }
}
.card {
  background: white; border-radius: 1.2rem;
  padding: 1.5rem; border: 1px solid #e2e8f0;
  box-shadow: 0 2px 6px rgba(0,0,0,0.03);
}
.card h2 {
  font-size: 1.2rem; margin-bottom: 1rem;
  border-left: 4px solid #3b82f6; padding-left: 0.7rem;
}

<!-- HTML after </header> -->
<div class="grid">
  <div class="card" id="left-panel">LEFT</div>
  <div class="card" id="right-panel">RIGHT</div>
</div>
```

Tip

Narrow your browser window past 700 px — both cards should stack vertically.

📋 Part 2 – Add-Note Form (Left Panel)

6 Build the form markup

Replace the placeholder text inside the first .card with the form below.

Note: we do NOT use a <form> element — plain s are fine here.

```
<div class="card" id="left-panel">
  <h2>✏️ Add a note</h2>

  <div class="form-group">
    <label>📌 Title *</label>
    <input type="text" id="noteTitle" placeholder="e.g., DOM practice">
  </div>

  <div class="form-group">
    <label>📝 Content</label>
    <textarea id="noteContent" rows="2"
              placeholder="Write something..."></textarea>
  </div>

  <button id="addNoteBtn">+ Add Note</button>
  <div class="counter" id="noteCounter">0 notes</div>
</div>
```

7 Style the form controls

Add these CSS rules for the label, input, textarea, button, and counter.

```
.form-group { margin-bottom: 1rem; }
label {
  font-weight: 600; font-size: 0.8rem;
  text-transform: uppercase; color: #475569;
  display: block; margin-bottom: 0.3rem;
}
input, textarea {
  width: 100%; padding: 0.7rem;
  border: 2px solid #e2e8f0; border-radius: 1rem;
  font-family: inherit; outline: none;
}
input:focus, textarea:focus { border-color: #3b82f6; }
button {
  background: #3b82f6; border: none; color: white;
  font-weight: 600; padding: 0.6rem 1.2rem;
  border-radius: 2rem; cursor: pointer;
  font-family: inherit; transition: 0.1s;
}
button:hover { background: #2563eb; }
```

```
.counter {
  background: #e6f0ff; display: inline-block;
  padding: 0.2rem 0.8rem; border-radius: 20px;
  font-size: 0.7rem; font-weight: 700; margin-top: 1rem;
}
```

Tip

The blue ring appears on input focus thanks to border-color: #3b82f6 in the :focus rule.

Part 3 – Notes List & BOM Info (Right Panel)

8 Build the right panel markup

Replace the RIGHT placeholder with this markup. It has three parts:

1) #notesContainer for the live list,

2) a BOM info box showing window size + user agent,

3) a "Clear all" button.

```
<div class="card" id="right-panel">
  <h2>📋 Your notes</h2>

  <div id="notesContainer" class="notes-list">
    <div style="color:#94a3b8;text-align:center;">
      ✨ No notes yet. Create one!
    </div>
  </div>

  <hr>
  <h2 style="font-size:1rem;">🌐 Browser info (BOM)</h2>
  <div class="info-panel" id="bomPanel">
    <div class="info-row">
      <span>🖥 Window size:</span>
      <span id="windowSize">-- x --</span>
    </div>
    <div class="info-row">
      <span>📱 User Agent:</span>
      <span id="userAgent">--</span>
    </div>
  </div>

  <button id="clearAllBtn"
          style="background:#64748b;margin-top:0.8rem;">
    🗑 Clear all notes
  </button>
</div>
```

9 Style the notes list and info panel

Add the CSS rules for .notes-list, .note-item, and the BOM info panel.

```
.notes-list {
  margin-top: 1.2rem; max-height: 380px;
  overflow-y: auto;
  display: flex; flex-direction: column; gap: 0.8rem;
}
.note-item {
  background: #f8fafc; border: 1px solid #e2e8f0;
  border-radius: 1rem; padding: 0.8rem 1rem;
}
.note-title {
  font-weight: 800;
  display: flex; justify-content: space-between;
  align-items: center; margin-bottom: 0.3rem;
}
.note-content { font-size: 0.85rem; color: #334155; }
.info-panel {
  background: #f1f5f9; border-radius: 1rem; padding: 0.8rem;
  margin-top: 1rem;
}
.info-row {
  display: flex; justify-content: space-between;
  font-size: 0.8rem; padding: 0.3rem 0;
  border-bottom: 1px dashed #cbd5e1;
}
.delete-note {
  background: #ef4444; padding: 0.3rem 1rem;
  font-size: 0.7rem; border-radius: 1rem;
}
.delete-note:hover { background: #dc2626; }
hr { margin: 1rem 0; }
```

Tip

The notes list gets a scrollbar automatically when notes overflow 380 px thanks to overflow-y: auto.

Part 4 – JavaScript: State & DOM References

10 Declare the in-memory state

Inside the <script> block, declare the notes array and nextId counter.

notes stores every note object; nextId gives each note a unique ID.

```
// ── STATE ──────────────────────────────────────────────
let notes  = [];   // each entry: { id, title, content }
let nextId = 1;
```

Tip

No localStorage is used — notes are lost on page refresh.
That's intentional for this lab. You can add persistence later as a bonus challenge!

11 Cache DOM elements in variables

Call getElementById once per element and store references in variables.

Re-using cached references is faster than searching the DOM repeatedly.

```
// ── DOM REFERENCES ──────────────────────────────────────
const notesContainer  = document.getElementById("notesContainer");
const titleInput      = document.getElementById("noteTitle");
const contentInput    = document.getElementById("noteContent");
const addBtn          = document.getElementById("addNoteBtn");
const clearAllBtn     = document.getElementById("clearAllBtn");
const noteCounterSpan = document.getElementById("noteCounter");
const windowSizeSpan  = document.getElementById("windowSize");
const userAgentSpan   = document.getElementById("userAgent");
```

Tip

Test in the browser console: type notesContainer and press Enter.
You should see the div element logged — confirming the reference works.

Part 5 – Helper Functions

12 Write updateCounter()

This helper updates the "X notes" badge whenever the notes array changes.

It uses a ternary for singular/plural: "1 note" vs "2 notes".

```
function updateCounter() {
  const count = notes.length;
  noteCounterSpan.textContent =
    count === 1 ? "1 note" : `${count} notes`;
}
```

Tip

You can test it in the console: notes.push({id:1,title:"hi",content:""}); updateCounter();

13 Write renderNotes() – clear the container

renderNotes() re-draws the entire list from scratch every time the array changes.

Step 1 (this step): safely clear the container using a while loop.

We use removeChild instead of innerHTML = "" to follow the lab constraint.

```
function renderNotes() {
  // ── clear container safely (no innerHTML) ──
  while (notesContainer.firstChild) {
    notesContainer.removeChild(notesContainer.firstChild);
  }

  // (rest of the function comes in the next step)
}
```

> ⚠ ⚠ **Important**
> Why not innerHTML = ""?
> The lab constraint bans innerHTML to practise "pure DOM" methods.
> removeChild() also avoids accidentally executing embedded scripts.

14 Write renderNotes() – empty state message

Inside renderNotes(), after clearing, check if notes is empty.
If it is, create a friendly placeholder div with createElement and textContent.

```
  // ── empty-state message ──
  if (notes.length === 0) {
    const emptyMsg = document.createElement("div");
    emptyMsg.textContent = "📭 No notes yet. Add your first note!";
    emptyMsg.style.color     = "#94a3b8";
    emptyMsg.style.textAlign = "center";
    emptyMsg.style.padding   = "1rem";
    notesContainer.appendChild(emptyMsg);
    updateCounter();
    return;
  }
```

15 Write renderNotes() – build each note item

After the empty-state check, loop through notes with forEach.
For each note, build the card DOM tree using only createElement, textContent, classList.add, setAttribute, and appendChild — no innerHTML.

```
  notes.forEach(note => {
    // wrapper div
    const noteDiv = document.createElement("div");
    noteDiv.classList.add("note-item");
    noteDiv.setAttribute("data-id", note.id);

    // title row
    const titleDiv  = document.createElement("div");
    titleDiv.classList.add("note-title");

    const titleSpan = document.createElement("span");
    titleSpan.textContent = note.title;
```

```
    // delete button
    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "🗑 Delete";
    deleteBtn.classList.add("delete-note");
    deleteBtn.setAttribute("data-action", "delete");
    deleteBtn.setAttribute("data-id",     note.id);

    titleDiv.appendChild(titleSpan);
    titleDiv.appendChild(deleteBtn);

    // content paragraph
    const contentP = document.createElement("p");
    contentP.classList.add("note-content");
    contentP.textContent = note.content || "(no additional content)";

    noteDiv.appendChild(titleDiv);
    noteDiv.appendChild(contentP);
    notesContainer.appendChild(noteDiv);
  });

  updateCounter();
}   // end renderNotes
```

Tip

The complete renderNotes() function now spans Steps 13 → 15.
Test it: add a dummy object to notes[] in the console, then call renderNotes().

🚀 Part 6 – Core Logic: Add, Delete, Clear

16 Write addNote() with BOM alert validation

addNote() reads the inputs, validates with BOM alert(), pushes to the array, calls renderNotes(), then clears and refocuses the title input.

```
function addNote() {
  const title = titleInput.value.trim();

  // BOM: alert() for empty-title validation
  if (title === "") {
    alert("❌ Title cannot be empty!");
    titleInput.focus();
    return;
  }

  const content = contentInput.value.trim();
  notes.push({ id: nextId++, title, content });

  renderNotes();

  // clear inputs after adding
  titleInput.value  = "";
```

```
  contentInput.value = "";
  titleInput.focus();
}
```

Tip

BOM alert() is a function of the window object (window.alert). We can write alert() for short.

17 Write deleteNoteById() with BOM confirm

deleteNoteById() asks the user to confirm before removing the note.

confirm() returns true (OK) or false (Cancel) — a classic BOM dialog.

We use Array.findIndex and splice to remove the note from the array.

```
function deleteNoteById(id) {
  // BOM: confirm() returns true/false
  const ok = confirm("⚠️ Are you sure you want to delete this note?");
  if (!ok) return;

  const index = notes.findIndex(n => n.id === id);
  if (index !== -1) {
    notes.splice(index, 1);
    renderNotes();
  }
}
```

Tip

confirm() is a BOM dialog — it pauses script execution until the user responds.
Modern apps use custom modal dialogs instead, but confirm() is great for learning.

18 Write clearAllNotes() with BOM confirm + alert

clearAllNotes() uses both confirm() and alert() — a double BOM demo.

It also resets nextId to 1 so IDs restart cleanly.

```
function clearAllNotes() {
  if (notes.length === 0) {
    alert("📭 No notes to clear.");
    return;
  }

  const ok = confirm(
    `❗ Delete ALL ${notes.length} notes? Cannot be undone.`
  );
  if (ok) {
    notes  = [];
    nextId = 1;
    renderNotes();
    alert("✨ All notes have been cleared.");
```

```
  }
}
```

Part 7 – Event Listeners & Delegation

19 Attach button click listeners

Attach click listeners to the Add Note and Clear All buttons.
Do this once during initialisation — not inside renderNotes().

```
addBtn.addEventListener("click", addNote);
clearAllBtn.addEventListener("click", clearAllNotes);
```

⚠ ⚠ Important
Never put addEventListener inside renderNotes().
renderNotes() runs every time the list changes, so you would attach duplicate listeners each time.

20 Use event delegation for delete buttons

Delete buttons are created dynamically so we cannot select them at page load.
Instead, listen on the parent #notesContainer and let click events bubble up.
We use closest() to find the button with data-action="delete".

```
notesContainer.addEventListener("click", (e) => {
  // find the closest ancestor (or self) that is a delete button
  const btn = e.target.closest('[data-action="delete"]');
  if (btn) {
    const id = parseInt(btn.getAttribute("data-id"), 10);
    deleteNoteById(id);
  }
});
```

Tip
Event delegation: instead of attaching a listener to every button,
attach ONE listener to the container. Clicks on children bubble up to it.
This works even for elements added after the page loaded.

Part 8 – BOM Features: Window Size & User Agent

21 Display window size with window.innerWidth/Height

Write updateWindowSize(). It reads window.innerWidth and window.innerHeight
and updates the #windowSize span with a template literal.

```
function updateWindowSize() {
  const w = window.innerWidth;
  const h = window.innerHeight;
  windowSizeSpan.textContent = `${w} x ${h} px`;
}
```

Tip

window.innerWidth/innerHeight are BOM properties — they describe the browser viewport, not the screen.

22 Listen for window resize

Attach a resize event listener to the window object so the display updates live.

Every time the user resizes the browser window, updateWindowSize() is called.

```
// BOM: resize event on the window object
window.addEventListener("resize", updateWindowSize);
```

Tip

Try resizing your browser window after this step — watch the numbers update in real time!

23 Read and display navigator.userAgent

Write showUserAgent(). It reads the navigator.userAgent string (a BOM property),

trims it to 55 characters for display, and sets the #userAgent span.

```
function showUserAgent() {
  let agent = navigator.userAgent;
  // trim long strings for display
  if (agent.length > 55) agent = agent.slice(0, 52) + "...";
  userAgentSpan.textContent = agent;
}
```

Tip

navigator is a BOM object. navigator.userAgent is a string identifying the browser.
It looks like: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit..."

Part 9 – Initialisation & Sample Notes

24 Add sample notes for a friendlier first load

Write addSampleNotes(). It pushes two demo notes so students see the UI populated

immediately — making it easier to test delete and clear features.

```
function addSampleNotes() {
  if (notes.length === 0) {
    notes.push({
      id: nextId++,
      title: "Hello BOM",
      content: "Try resizing the window → watch width/height update!"
    });
    notes.push({
      id: nextId++,
      title: "Delete me",
      content: "Click delete and confirm the BOM confirm box."
    });
    renderNotes();
  }
}
```

25 Write init() and call it

Tie everything together in an init() function called once on page load.

Order matters: render the list before setting up event listeners.

```
function init() {
  addSampleNotes();     // populate demo notes
  updateWindowSize();   // BOM: show current size
  showUserAgent();      // BOM: show user agent
  addBtn.addEventListener("click", addNote);
  clearAllBtn.addEventListener("click", clearAllNotes);
}

init();   // ← call init when the page loads
```

Tip

All addEventListener calls live in init(). This keeps set-up code in one clean, readable place.

Part 10 – Footer, Test Checklist & Bonus Ideas

26 Add the footer

After the .grid div, inside .container, add a small footer.

This is purely decorative — no JS needed.

```
<!-- HTML after </div> .grid -->
<footer>
  BOM: confirm() on delete · alert() on empty title
    · resize event updates window size.
</footer>
```

```
/* CSS */
footer {
  text-align: center; margin-top: 2rem;
  font-size: 0.7rem; color: #5b6e8c;
}
```

27 Final test checklist

Open index.html and verify each feature:

- [x] Two sample notes appear on load.
- [x] Counter shows "2 notes".
- [x] Window size displays (e.g. 1280 x 720 px).
- [x] User agent string is visible (truncated).
- [x] Resizing the browser updates the size live.
- [x] Clicking "+ Add Note" without a title shows alert().
- [x] Adding a valid note appends it to the list.
- [x] Counter increments correctly.
- [x] Delete button shows confirm() dialog.
- [x] Confirming delete removes the note and updates counter.
- [x] "Clear all notes" shows confirm(); then alert() on success.
- [x] After clearing, empty-state message appears.

28 Bonus: Add prompt() to rename a note

Try this challenge: add an "Edit" button beside "Delete".

When clicked, use BOM prompt() to ask for a new title, then update the note.

```
// In renderNotes(), create the edit button:
const editBtn = document.createElement("button");
editBtn.textContent = "✎ Edit";
editBtn.setAttribute("data-action", "edit");
editBtn.setAttribute("data-id", note.id);

// In the event delegation listener, handle "edit":
const editTarget = e.target.closest('[data-action="edit"]');
if (editTarget) {
  const id    = parseInt(editTarget.getAttribute("data-id"), 10);
  const found = notes.find(n => n.id === id);
  if (found) {
    // BOM: prompt() returns new string or null
    const newTitle = prompt("New title:", found.title);
    if (newTitle !== null && newTitle.trim() !== "") {
      found.title = newTitle.trim();
      renderNotes();
    }
  }
```

```
}
```

Tip

prompt(message, defaultValue) is the third BOM dialog — it returns a string or null.
null means the user clicked Cancel.

29 Bonus: Add keyboard shortcut (Enter to add)

Allow students to press Enter in the title field to add the note.
Use keydown event on titleInput and check e.key.

```
titleInput.addEventListener("keydown", (e) => {
  if (e.key === "Enter") {
    addNote();
  }
});
```

Tip

e.key === "Enter" is more readable than the old e.keyCode === 13.
Add this line inside init() along with the other event listeners.

30 Bonus: Show a screen-size category with BOM

Extend updateWindowSize() to also classify the screen as Mobile / Tablet / Desktop based on window.innerWidth breakpoints — a real-world BOM use case.

```
function updateWindowSize() {
  const w = window.innerWidth;
  const h = window.innerHeight;
  let category;
  if (w < 480)       category = "📱 Mobile";
  else if (w < 1024) category = "💻 Tablet";
  else               category = "🖥️ Desktop";
  windowSizeSpan.textContent = `${w} x ${h} px  (${category})`;
}
```

Tip

Resize the browser window slowly — watch the label switch between Mobile, Tablet, and Desktop!